



Binary Trees.

[COMMENT] Complete the Table, Continue the correctness of the minimal length.

Arrays, Linked Lists, Heaps, and Hash Tables are all ways to maintain collections. Each offers a different trade-off in query running times. For some of them, we imposed good worst-case bounds, while for others we can only compute expected costs (Hash Tables), detail are supplied in Table 5.1. The obvious question is whether one could guarantee both fast access and update running times in the worst-case.

In the following two lectures, we will see that using Balanced Binary Search Trees, one can provide both access and insertion/deletion in $O(\log n)$.

Data Structure	Access	Max/min	Insert/Delete	Max/min
Array	Data 2	Data 3	Data 3	$O(\log n)$
Linked-List	$O(n)$	$O(n)$	$O(1)$	$O(\log n)$
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	$O(n)$	$O()$	$O()$	$O(\log n)$

Table 5.1: Running times respecting to data structure and task

1 Binary Trees and How to Encode Them

We have already seen, in heaps, that organizing our data in a graph-like structure can offer a speed advantage. For future applications, and in particular for maintaining data in sorted order, we will have to encode our data using binary trees. In contrast to heaps, these trees may not be almost complete and also have to support pointer manipulations, specifically placing a binary tree as a left or right subtree of a given node. To enable this, we will have to treat the **right**, **left**, and **parent** as variables, in contrast to heaps where they are determined completely by the node index. We begin this section by stating definitions.

Definition 1.

1. Binary Tree: A tree in which any vertex has at most two children.

2. A descendant of vertex x is a vertex in the subtree whose root is x . A left descendant of vertex x is either a vertex in the subtree whose root is the left child of x , or x 's left child.
3. An ancestor of x is a vertex to which x belongs as a descendant.
4. A leaf is a vertex without children.
5. Height of vertex x is the length of the longest simple path (without cycles) between x and one of the leaves. The height of leaf is 0.
6. Height of the tree is the height of its root, which is usually denoted by h .

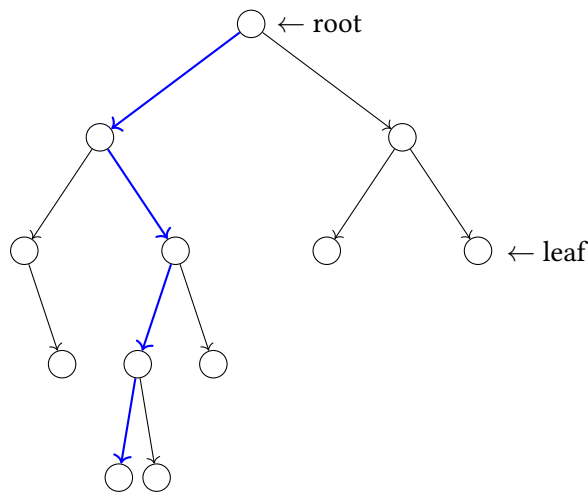


Figure 5.1: Example of a binary tree with 12 vertices. In blue, one of the longest paths from the root to one of the leaves, which is of length 4.

Question. What are the maximal and minimal heights of binary trees over n vertices?

Answer. The maximal height is $n-1$, and matches to trees in which any vertex has at most one child. The minimal height is $\log n$. Let's prove it by induction.

Denote by H_n the minimal height of a binary tree with n vertices. Observe that $H_{n+1} \geq H_n$. That's because adding a vertex to an n -size binary tree can't shorten any of the paths from root to leaves, particularly not the longest one.

1. Base. $n = 1$, The tree consists of a single vertex, which is both the root and a leaf. Thus, the longest path is of length 0, and also $\log 1 = 0$.
2. Assumption. Assume the correctness of the claim for any $m < n$.
3. Step. Consider a binary tree over n vertices with minimal height. The longest path consists of an edge between the root and one of its children. Denote

by x the size of the root's left subtree, so the size of root's right subtree is $n - x$. Thus we get:

$$H_n \geq 1 + \max(H_x, H_{n-x})$$

By the monotonic property of H_n , we have that the above is minimized when both $H_x = H_{n-x} = H_{n/2}$ if n is even, or $\max(x, n-x) = \lceil n/2 \rceil$ when n is odd. So we get:

$$\begin{aligned} &\geq 1 + H_{\lceil n/2 \rceil} \\ &= 1 + \log(\lceil \frac{n}{2} \rceil) = \log(n+1) \end{aligned}$$

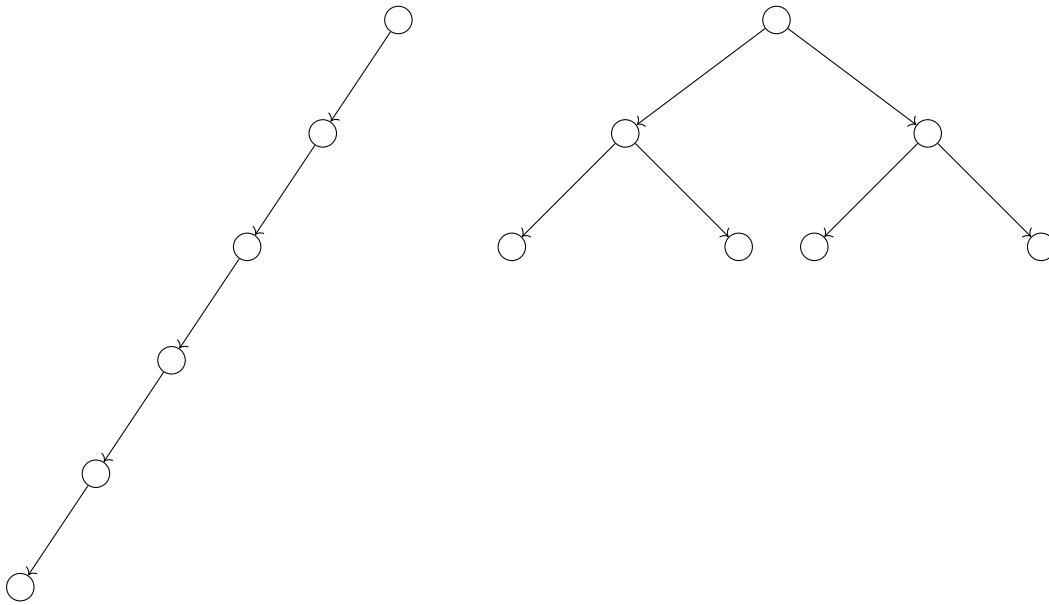


Figure 5.2: On the left, a binary tree with 6 vertices at height 5, while on the right, a binary tree with 7 vertices at height 2.

1.1 Encoding

We encode a binary tree by associating a field to each vertex x , representing its right, left children, and parent. We use the notation $x.\text{left}$ to refer to the left child of x , although the physical implementation may differ conceptually. For example, the way binary trees are implemented in Cormen is through 4 arrays. The first stores the value of x , while the others store pointers of specific types. For instance, the array `LEFT`, where `LEFT. $x$` stores the left child of x .

If nothing else has been mentioned, then we can assume that we can add additional fields to the vertices.

2 Binary Search Trees.

Now, we are ready to understand to store numbers (or keys), by binary trees.

Definition 2. Binary search tree (BST) is a binary tree which any node x of it:

1. Contains a field key, storing a number $x.\text{key}$.
2. Any left descendant y of x satisfies $y.\text{key} \leq x.\text{key}$.
3. Any right descendant y of x satisfies $y.\text{key} \geq x.\text{key}$.

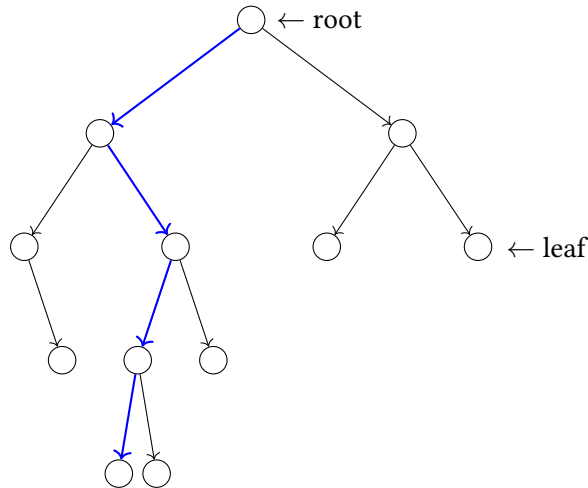


Figure 5.3: The heap after the exchange, at line (9). The induction assumption is applied over the right subtree, which is framed in gray.

Question. Let T be a binary search tree, Where are the minimum and maximum values of T ? (most left and right nodes).

Answer. Consider the algorithm which gets down by taking only left turns until hitting edge, as in Algorithm 1, Guideline for correctness: denote by z, r the minimal vertex by both value and height, or in other words, the highest vertex among the vertices with minimal value, and the root. Consider the path descending from r to z . Suppose that while walking through the path, we take a right turn at vertex y . Then, by the BST property, $y \leq z$. If $y < z$, then it contradicts z being minimal by value. Otherwise, if $y = z$, then it contradicts z being the highest vertex among the minimal vertices by value.

Definition 3. Let T be a binary search tree, and let x be a node belonging to it.

1. The **predecessor** of x will be defined as a vertex y such that $y.\text{key} \leq x.\text{key}$ and $y.\text{key}$ is maximal among the nodes satisfying this condition. If we were to set the values of T in sorted order, then the predecessor of x would be located on its left.

Data: T - tree

Result: pointer to vertex with minimum value in T

```

1 if  $T$  is empty then
2   | return null
3 end
4 if  $T.left$  is empty then
5   | return  $T$ 
6 end
7 return Min( $T.left$ )

```

Algorithm 1: Min

2. The **successor** of x will be defined as y , where x is the predecessor of y .

Functionality of BST.

1. Search(T , key): returns a pointer to the vertex whose key equals key.
2. Min(T): returns a pointer to the vertex with the minimum value in T .
3. Max(T): returns a pointer to the vertex with the maximum value in T .
4. Predecessor(x): returns a pointer to the predecessor of x .
5. Successor(x): returns a pointer to the successor of x .
6. Insert(T , key): inserts key into T (creates a new vertex).
7. Delete(T , x): removes x from T .
8. Inorder(T): outputs T 's keys in sorted order.

3 Appendix - BST Methods Source: A Mixture of Cormen, Wikipedia, and Codeforces

Data: T - tree, key - key to search for

Result: pointer to vertex with key equal to key

```

1 if  $T$  is empty then
2   | return null;
3 end
4 if  $T.key$  equals key then
5   | return  $T$ ;
6 end
7 if key is less than  $T.key$  then
8   | return Search( $T.left$ , key);
9 end
10 else
11   | return Search( $T.right$ , key);
12 end
```

Algorithm 2: Search

Data: x - vertex

Result: pointer to predecessor of x

```

1 if  $x.left$  is not empty then
2   | return Max( $x.left$ );
3 end
4  $y \leftarrow x.parent$ ;
5 while  $y$  is not empty and  $x$  is  $y.left$  do
6   |  $x \leftarrow y$ ;
7   |  $y \leftarrow y.parent$ ;
8 end
9 return  $y$ ;
```

Algorithm 3: Predecessor

Data: x - vertex
Result: pointer to successor of x

```
1 if  $x.right$  is not empty then  
2   |   return Min( $x.right$ );  
3 end  
4  $y \leftarrow x.parent$ ;  
5 while  $y$  is not empty and  $x$  is  $y.right$  do  
6   |    $x \leftarrow y$ ;  
7   |    $y \leftarrow y.parent$ ;  
8 end  
9 return  $y$ ;
```

Algorithm 4: Successor